Reinforcement Learning for Pseudo-Labeling

Capstone Project Presentation

Group-11

Students: Aishwarya Maddula, Deepika Reddygari, Phanindra Kalaga, Prudhvi Chekuri

Guided by: Dr. Amir Jafari, Prof. Tyler Wallett

George Washington University

Outline

1. Introduction
2. Background and Literature Review
3. Reinforcement Learning Framework
4. Training Flow
5. Project Setup
6. PseudoCode
7. Implementation Details
8. Results and Insights
9. Discussion
10. Conclusion
11. Limitations
12. Future Scope

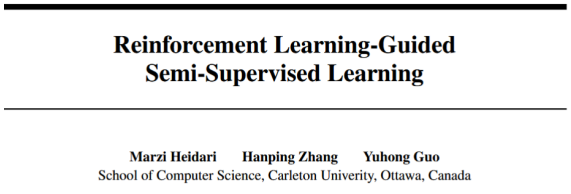
Introduction

- **Problem:** Labeled data is limited and costly to obtain.
- **Challenge:** Traditional pseudo-labeling often propogates errors.
- **Idea:** Use Reinforcement Learning (RL) to adaptively select pseudo-labels and improve error rate.

Background and Literature Review

- The team began with introductory RL implementations using examples from the `gymnasium` library to build foundational understanding.

- Conducted an initial review and implementation of five RL-based pseudo-labeling studies to establish a research baseline.



Reinforcement Learning-Guided Semi-Supervised Learning

Marzi Heidari Hanping Zhang Yuhong Guo
School of Computer Science, Carleton Univerity, Ottawa, Canada



Selective Pseudo-Labeling with Reinforcement Learning for Semi-Supervised Domain Adaptation

Bingyu Liu[1,3], Yuhong Guo[1,2], Jieping Ye[1], Weihong Deng[3]
[1] Didi Chuxing [2] Carleton University [3] Beijing University of Posts and Telecommunications

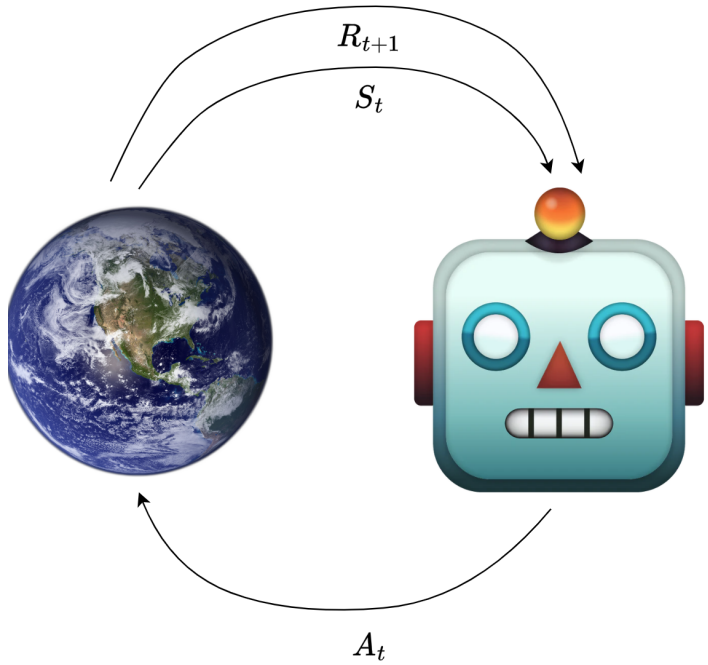


A Review of Pseudo-Labeling for Computer Vision

Patrick Kage* P.KAGE@ED.AC.UK
School of Informatics, The University of Edinburgh, Edinburgh, UK
Jay C. Rothenberger* JAY.C.ROTHENBERGER@OU.EDU
School of Computer Science, The University of Oklahoma, Norman OK, USA
Pavlos Andreadis PAVLOS.ANDREADIS@ED.AC.UK
School of Informatics, The University of Edinburgh, Edinburgh, UK
Dimitrios I. Diochnos DIOCHNOS@OU.EDU
School of Computer Science, The University of Oklahoma, Norman OK, USA

Reinforcement Learning Introduction

- **Reinforcement Learning:** Agent learns through interaction and feedback from the environment.
- **Sequential Decision Making:** Each action affects future states and rewards.
- **Markov Decision Process (MDP):** Framework for modelling decision making.



Markov Decision Process

MDP Formulation

State

Image features, model predictions, entropy (uncertainty).

$$\mathcal{S} = \{\mathbf{X};\ \text{softmax}(\mathbf{y});\ \mathcal{L}\}$$

Action

Assign pseudo-label or skip.

$$\mathcal{A} = \{0, \ldots, 9, ;\ \text{skip}\}$$

Reward

Error Rate improvement of downstream model on validation data.

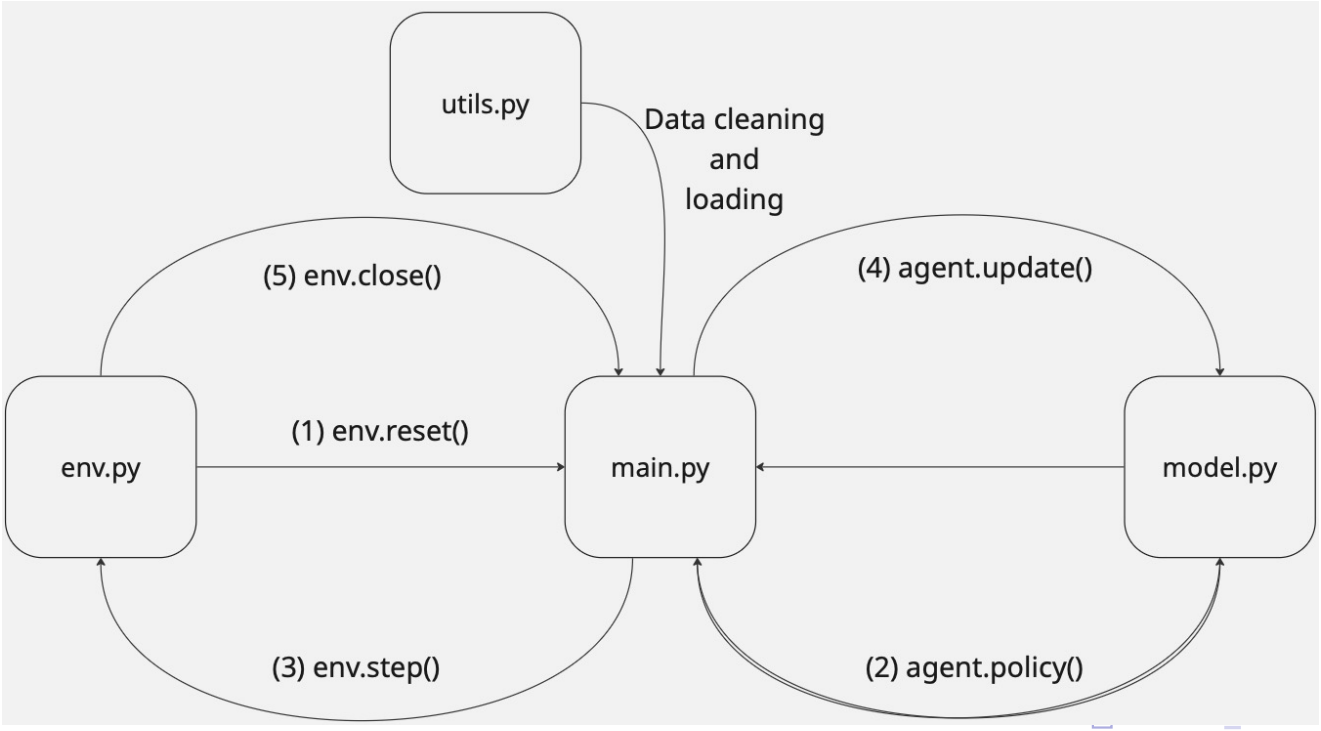
$$\mathcal{R}_t = -(e_t - e_{t-1})$$

Workflow

1. **Load** labeled and unlabeled MNIST data.
2. **Initiate** RL environment and agent.
3. RL agent selects **pseudo-label** actions.
4. **F**or RLGSSL: generate mixup samples, compute consistency reward, update classifier/policy jointly.
5. **Retrain** downstream model with newly labeled data.
6. **Reward** is assigned based on downstream model's error rate.

Code Files

- **main.py**: Entry point for training process.
- **env.py**: Custom RL environment handling pseudo-labeling episodes.
- **model.py**: Updates policy based on agent's actions for maximising reward.
- **utils.py**: Helper functions for data loading, metrics, etc.



PseudoCode

Algorithm RL agent training loop

Require: state_dim, action_dim, labeled_data, unlabeled_data, num_episodes
  agent ← `RLAgent`(state_dim, action_dim)
  env ← `PseudoLabelEnv`(labeled_data, unlabeled_data)

  for episode = 1 **to** num_episodes **do**
    state ← env.reset()
    done ← false
    while not done **do**
      action ← agent.select_action(state)
      (next_state, reward, terminated, info) ← env.step(action)
      state ← next_state
      done ← terminated
    end while
    agent.update()
    accuracy ← env.evaluate()
  end for
  env.close()

env.reset()

```
def reset(self, *, seed=None, options=None):
    super().reset(seed=seed)
    self.current_idx = 0
    self.newly_pseudo_labeled_data = []
    self.episode_reward = 0.0

    perm = torch.randperm(self.unlabeled_x.size(0))
    self.unlabeled_x = self.unlabeled_x[perm]
    self.unlabeled_y_true = self.unlabeled_y_true[perm]

    self.baseline_accuracy = self.downstream_model.evaluate_model(self.val_x, self.val_y)

    return self._get_state(), {}
```

- **Input:** Random seed and options.
- **Functionality:** Resets the environment.
- **Output:** Initial environment state and empty info dictionary.

model.policy()

```
def policy(self, state):
```

```
    with torch.no_grad():
        state_tensor = torch.FloatTensor(state).unsqueeze(0).to(self.device)
        logits = self.pi(state_tensor)
        action_dist = torch.distributions.Categorical(logits=logits)
        action = action_dist.sample()
        return action.item()
```

- **Input:** Current state (S_t).
- **Functionality:** Action (pseudolabel) is selected based on logits obtained from policy network.

$$a \in \mathcal{A}(s)$$

- **Output:** Selected action as integer.

env.step()

```python
def step(self, action):
    img_idx = self.current_idx
    reward = 0
    if action < self.num_classes:
        true_label = self.unlabeled_y_true[img_idx]
        preds = self.downstream_model.get_predictions(self.unlabeled_x[img_idx].unsqueeze(0))
        pred_label = torch.argmax(preds).item()
        reward = self.calculate_reward(pred_label, true_label, action, preds)
        self.episode_reward += reward
        self.newly_pseudo_labeled_data.append(
            (self.unlabeled_x[img_idx], torch.tensor(action, device=self.device))
        )
    else:  # Action is skip (action == num_classes)
        reward = self.calculate_reward(None, None, action, None)

    self.current_idx += 1
    terminated = (self.current_idx >= len(self.unlabeled_x))
    info = {}
    if terminated:
        self.downstream_model.train_model((self.labeled_x, self.labeled_y), self.newly_pseudo_labeled_data)
        new_accuracy = self.downstream_model.evaluate_model(self.val_x, self.val_y)
        info['new_accuracy'] = new_accuracy
        accuracy_bonus = new_accuracy - self.baseline_accuracy
        reward += accuracy_bonus
        info['final_reward'] = self.episode_reward + accuracy_bonus

    next_state = self._get_state()
    return next_state, reward, terminated, False, info
```

env.step()

- **Input:** Action selected by the agent.
- **Functionality:** There are multiple functionalities like:
  - Computes reward based on prediction and action.
  - Updates pseudo-labeled data.
  - Trains downstream model at episode end.
- **Output:** Returns next state, total reward, termination flag, and performance info (error rate, final reward).

model.update()

```
def update(self, sampled_states, sampled_actions, sampled_rewards):
    # Compute advantages
    rewards_to_go, values, adv = self.compute_advantage(sampled_states, sampled_rewards)

    # Convert to tensors
    states_tensor = torch.FloatTensor(np.array(sampled_states)).to(self.device)
    actions_tensor = torch.LongTensor(sampled_actions).to(self.device)

    # Update policy network
    self.optimizer_pi.zero_grad()
    logits = self.pi(states_tensor)
    log_probs = F.log_softmax(logits, dim=1)
    action_log_probs = log_probs.gather(1, actions_tensor.unsqueeze(1)).squeeze(1)
    policy_loss = -torch.mean(action_log_probs * adv.detach())
    policy_loss.backward()
    self.optimizer_pi.step()

    # Update value network
    self.optimizer_v.zero_grad()
    value_preds = self.v(states_tensor).squeeze(1)
    value_loss = torch.mean((rewards_to_go - value_preds) ** 2)
    value_loss.backward()
    self.optimizer_v.step()

    return policy_loss.item(), value_loss.item()
```

model.update()

- **Input:** Sampled states, actions, and rewards from agent experience.
- **Functionality:** There are multiple functionalities like:
  - Computes advantages using rewards-to-go and value estimates.
  - Updates policy network ($\pi_\theta(a|s)$) to maximize expected reward.
  - Updates value network (V_θ^π) to minimize prediction error.
- **Output:** Returns policy loss and value loss for performance tracking.

main.py (Bringing everything together)

```python
def main(EPISODES, NUM_LABELED, NUM_UNLABELED, NUM_VALIDATION):
    env = PseudoLabelEnv(NUM_LABELED=NUM_LABELED, NUM_UNLABELED=NUM_UNLABELED, NUM_VALIDATION=
      NUM_VALIDATION)
    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n
    agent = PolicyGradient(state_dim=state_dim, num_actions=action_dim)
    for episode in tqdm(range(EPISODES), desc="Training Progress"):
        state, info = env.reset()
        terminated = False
        sampled_states, sampled_actions, sampled_rewards = [], [], []
        while not terminated:
            action = agent.policy(state)
            next_state, reward, terminated, _, info = env.step(action)

            # Store the experience
            sampled_states.append(state)
            sampled_actions.append(action)
            sampled_rewards.append(reward)

            state = next_state

        agent.update(np.vstack(sampled_states), sampled_actions, sampled_rewards)

    env.close()
```

Key Observations

- RL-based pseudo-labeling improves early-stage learning performance.
- CNN benefits more from RL guidance than ResNet due to weaker baseline.
- Performance degradation observed after multiple episodes.
- **Key Bottleneck:** Reward design and confirmation bias.

Model	**MNIST (%)**	**CIFAR-10 (%)**
CNN Baseline	96.55	65.60
CNN + RL	**98.12**	**68.40**
ResNet Baseline	98.76	74.10
ResNet + RL	98.12	73.80

Final Classification Accuracy Comparison

What We Learned Since Midterm

- RL improves pseudo-label selection primarily in early epochs.
- Performance degradation occurs due to error reinforcement.
- Reward shaping has higher impact than network architecture.
- CNN shows stronger gains from RL guidance than ResNet.
- RL training is significantly more computationally expensive.

Discussion

- RL dynamically adapts pseudo-label selection beyond static confidence thresholds.
- Early performance gains confirm RL's potential for label efficiency.
- Degradation highlights the need for long-term reward stabilization.
- ResNet stability limits the impact of RL compared to CNN.

Conclusion

- Successfully demonstrated RL-based pseudo-labeling in semi-supervised learning.
- RL improves early learning by selecting high-confidence unlabeled samples.
- Long-term training stability remains a critical challenge.
- Reward design plays a central role in RL-based pseudo-labeling.
- The framework shows strong research potential but needs stability optimization.

Limitations

- **Performance Degradation**: Accuracy drops after several episodes due to confirmation bias.
- **Reward Sensitivity**: Small reward changes cause unstable training.
- **Computational Cost**: RL training is 3–4× slower than standard pseudo-labeling.
- **Noisy Labels**: Early incorrect pseudo-labels propagate errors.

Future Scope

- Implement PPO and Trust Region Policy Optimization for improved stability.
- Introduce uncertainty-aware and entropy-calibrated rewards.
- Apply curriculum learning by delaying hard samples.
- Extend to domain adaptation and real-world noisy datasets.
- Add visualization dashboards for reward and policy tracking.

Thank You

Questions?